

Periodic Task Set 產生方式說明

基礎參數分層與 Workload Density 釐定

我們將 Period 分配為前半部固定為 6，中段為 11 或 12，最後兩個為 15、18、21 或 24。Execution time 則確保至少有 2 個 e=2，以及 2 個 e=3。此分層設計旨在將整體系統的 Workload Density (DW) 精準釐定在 0.75 至 1.0 之間，避免 DW 過低導致排程無挑戰性，或 DW 大於 1.0 造成理論過載而無法產生可行解，從而保留最佳的排程彈性與 Slack 空間。

防禦性 Deadline 與 Frame Size 數學對齊

Clock-Driven 排程要求 Frame Size (f) 必須滿足 $2f - \gcd(f, \text{period}) \leq \text{deadline}$ 。在我們的設計中，最後兩個任務的 Period 為 15 至 24，且 Deadline 強制設為 3（等於最大 Execution time）。代入公式驗證，唯有 $f = 3$ 能完美通過所有限制式。因此，我們的生成器會自動驗證並鎖定 $f = 3$ ，確保排程器在離線階段即具備數學可行性，避免寫死參數導致的評分風險。

Non-preemptive 任務設計

作業規定需包含 Non-preemptive 任務。初期設計擬將最長執行時間 (e=3) 綁定為不可中斷，但經分析發現，e=3 且 d=3 會產生零鬆弛 (Zero-slack) 的連續區塊，極易導致排程碎片化與求解震盪。最終我們改為綁定 e=2 的任務為 Non-preemptive，在滿足規範的前提下，顯著降低了排程演算法的複雜度與 Acceptance Test 的拒絕率。

Sporadic 與 Aperiodic Task 生成策略與壓力測試設計

為了壓力測試 Acceptance Test 的魯棒性與系統在突發負載下的 Slack 保留策略，我們開發了獨立的生成腳本，並引入以下額外設計：

固定 Seed 與情境多樣性

我們固定基礎隨機種子 (Base Seed = 2026)，確保每次執行與 Demo 結果 100% 可重現。腳本自動生成 10 筆情境，分為 3 筆 Uniform (均勻分佈)、3 筆 Concentrated (集中分佈) 與 4 筆 Mixed (混合分佈)，以模擬真實電網中不同型態的突發與常態負載。

分佈類型設計與壓力測試目的

- Uniform 分佈**：釋放時間均勻分佈於 0 至 72-e，用於測試系統在一般隨機負載下的基礎接納率與 Slack 消耗速度。
- Concentrated 分佈**：70% 的任務集中在特定尖峰時段（如 0-5, 8-13, 18-23），且伴隨較高的能源需求。此設計旨在測試系統在極端擁塞與能源緊繃狀態下，Acceptance Test 是否能正確拒絕不可行的 Sporadic 任務，並保護既有 Periodic 任務不違約。
- Mixed 分佈**：30% 集中加上 70% 均勻，貼近真實電網的混合負載情境，用於評估系統在動態變化中的權衡能力與 Reserve 保留策略的有效性。

AI 輔助開發說明

我們先根據作業限制式（如 Frame Size 公式、DW 範圍）進行數學計算與邊界分析，撰寫出核心參數分配的 Pseudocode 後，再交由 AI 生成完整的 Python 實作程式碼與驗證腳本。

核心參數分配 Pseudocode：

```
task_num = rand(6, 18)

# Period 分層: 前半部固定為6, 中段11或12, 末段長週期
period = [6, ..., (11~12), (15/18/21/24)]
# 數量分配: ((Jp-2)/2 or +1) 個 6, ((Jp-2)/2) 個 11/12, 最後 2 個長週期

# Execution time: 確保滿足作業規範 (至少2個e=2, 至少1個e=3)
e = [1, ..., 2, 2, 3, 3]

# Deadline: 前半部保留排程彈性, 末段強制 d=e 以對齊 Frame Size f=3
deadline = [rand(2*max_e, p_j), ..., max_e, max_e]

# Release time: 隨機分佈於週期內
r = [rand(1, p_j)]

# Energy demand: 確保至少2個高耗能任務 (w >= 14)
w = [rand(6, 14), rand(14, 18), rand(14, 18)]
```

虛擬電廠 (VPP) Level 2 進階動態排程系統架構

一、模型符號定義 (Notation)

符號 (Symbol)	說明 (Description)	單位 (Unit)
T	排程週期總長度 (預設為 72)	h
t	時間索引, $t = 0, 1, \dots, 71$	—
R, G, B	分別代表「再生能源機組」、「火力機組」與「電池」之集合	—
J^P, J^S, J^A	分別代表 Periodic、Sporadic (硬限制) 與 Aperiodic (軟限制) 任務集合	—
w_j	任務 j 每小時所需要的電能需求	MWh
$x_{j,t}$	二元決策變數, 代表任務 j 在時間 t 是否被執行 (0 或 1)	—
$\hat{R}_{r,t}$	再生能源 r 在時間 t 的 預測出力比例 (0 ~ 1)	—
$R_{act,r,t}$	再生能源 r 在時間 t 的 實際出力	MWh
$\varepsilon_{r,t}$	該時間點的預測隨機誤差	—
ρ	最大預測誤差比例邊界 (Forecast Error Ratio)	—
cap_r	再生能源 r 的實體裝置容量	MWh
$SOC_{b,t}$	電池 b 在時間 t 的荷電狀態 (State of Charge)	MWh
SOC_b^{min}, SOC_b^{max}	電池 b 的安全最低與最高 SOC 容量限制	MWh
$\eta_b^{chg}, \eta_b^{dis}$	電池 b 的充電與放電轉換效率 (0 ~ 1)	—
σ_b	電池 b 的每小時自放電率	1/h
$C_{b,t}$	電池 b 在時間 t 從電網吸取的充電輸入量	MWh
$D_{b,t}$	電池 b 在時間 t 真正提供給負載的放電輸出量	MWh
deg_b	電池 b 每 MWh 放電的老化成本	\$/MWh
$Commit_t$	虛擬電廠在時間 t 的日前承諾售電量	MWh
$Sell_t$	時間 t 最終實際賣給電網的總售電量	MWh
π_t	時間 t 的一般市場電價	\$/MWh
λ^{rt}	即時市場高電價套利倍率 (Real-time Price Multiplier)	—
ρ^{pen}	未達承諾的售電缺口罰款費率	\$/MWh
$Penalty_t$	時間 t 的市場違約總罰金	\$

二、Level 2 架構簡介與放寬假設

相較於基礎的排程，Level 2 系統大幅放寬了理想假設，將運作環境推向更接近真實世界的虛擬電廠情境：

- 綠電預測具備隨機性**：實際發電量不再等同於完美預測，而是帶有隨機誤差。
- 真實的電池物理模型**：電池不再是完美的儲能容器，引入了充放電效率損失、每小時自放電率，以及隨電量遞減的動態放電極限。
- 具備違約風險的電力市場**：系統必須面對日前售電承諾，若因缺電違約將面臨重罰；反之，若在市場高價時超額發電，則能獲取套利利潤。

三、Level 2 核心限制式 (Constraints)

系統的運作嚴格遵守以下 16 條物理與市場限制式：

【綠電不確定性模型】

1. 實際再生能源出力

實際出力等於預測值乘上隨機誤差因子。

$$R_{act,r,t} = cap_r \times \hat{R}_{r,t} \times (1 + \varepsilon_{r,t})$$

2. 實際出力上下界

實際發電量不得為負值，且絕對不可能超過機組的物理裝置容量。

$$0 \leq R_{act,r,t} \leq cap_r$$

3. 預測誤差上界

隨機誤差被嚴格控制在參數範圍內（例如正負 20%），不得無限肆意發散。

$$|\varepsilon_{r,t}| \leq \rho$$

【真實電池物理防護模型】

4. 含效率的狀態轉移

放電時電池內部消耗的電量大於輸出至電網的電量；充電時電池增加的電量小於從電網抽取的電量。

$$SOC_{b,t+1} = SOC_{b,t} - \frac{D_{b,t}}{\eta_b^{dis}} + C_{b,t} \times \eta_b^{chg} - SelfDischarge_t$$

5. 電量絕對上下界

電池的即時電量 (SOC) 永遠被限制在安全上限與下限之間。

$$SOC_b^{min} \leq SOC_{b,t} \leq SOC_b^{max}$$

6. 充電功率限制

每小時充入的電量不得超過逆變器的物理充電上限，且受限於電池剩餘可用空間。

$$C_{b,t} \leq \min \left(C_b^{max}, \frac{SOC_b^{max} - SOC_{b,t}}{\eta_b^{chg}} \right)$$

7. 放電功率限制

每小時輸出的交流電不得超過逆變器的物理放電上限。

$$0 \leq D_{b,t} \leq D_b^{max}$$

8. 具底線保護的自放電

電池隨時間流逝會固定損耗電量，但設有安全防護，自放電絕對不會讓電池跌破安全下限。

$$SOC_{b,t+1} = \max \left(SOC_b^{min}, SOC_{b,t} \times (1 - \sigma_b) \right)$$

9. 雙重放電防護極限

電池的實際放電極限受制於「逆變器隨電量降低而遞減的極限」與「內部剩餘化學能轉換後的物理極限」，兩者取其低，確保能量嚴格守恆。

$$D_{b,t}^{eff} = \max \left(0, \min \left(D_b^{max} \times \min \left(1, \frac{SOC_{b,t} - SOC_b^{min}}{0.2 \times (SOC_b^{max} - SOC_b^{min})} \right), (SOC_{b,t} - SOC_b^{min}) \eta_b^{dis} \right) \right)$$

【市場機制與緊急排程】

10. 售電承諾缺口

若實際賣給電網的電量低於日前承諾的售電量，即產生違約缺口。

$$Shortfall_t = \max(0, Commit_t - Sell_t)$$

11. 市場違約罰金

針對售電缺口，系統會依據高昂的罰款費率扣除營收。

$$Penalty_t = Shortfall_t \times \rho^{pen}$$

12. 硬限制任務保護

在極端缺電危機時，系統絕對不中斷具有硬截止期限 (Hard-Deadline) 的任務，只能將可延遲 (Soft-Deadline) 的 Aperiodic 任務強行中斷。

$$x_{j,t} = 1, \quad \forall j \in J^P \cup J^S$$

13. 即時超賣套利收益

如果當前實際發電量大於售電承諾，多出來的電能可以按照即時市場的高電價倍率賣出，獲取超額利潤。

$$ExtraRev_t = \max(0, Sell_t - Commit_t) \times \pi_t \times (\lambda^{rt} - 1)$$

14. 充電虛擬任務追蹤

為了維持能量守恆，電池充電行為在能量流圖中會被視為一個虛擬耗電任務來結算。

$$x_{b,chg,t} = 1 \Rightarrow C_{b,t} > 0$$

15. 延遲狀態完美復原

被系統強行中斷的 Aperiodic 任務絕對不能憑空消失，必須精準復原其剩餘執行時間並塞入等候佇列擇日再戰。

$$j \in J^A \wedge j \text{ deferred at } t \Rightarrow j \in Queue_{t+1}^A$$

16. 修正後目標函數

Level 2 的總目標值完整包含了「火力成本」、「電池老化成本」、「市場違約罰款」，並扣除「基礎售電收入」與「即時套利超額收入」。

$$Obj^{J2} = GenCost + BatteryDegCost + Penalty - MarketRev - ExtraRev$$

排程演算法設計說明：日前排程與保留

- 日前排程方式 (**offline_planner**): 採用depth first search, 將每一條可行的路徑都走過一遍, 直到找出可行解。
 - 實作細節
 - 可行性判定: 根據Periodic task所推算出的frame size, 我們將72小時分為24個frame開始進行DFS, 在每一個frame的開頭將遍歷所有當前可執行的task的排列組合 (主要考量為發電機組的量能), 只要一找到當前可行的組合, 就會為當前的發電機組與電池進行快照 (紀錄當前數據), 並遞迴進入下一個frame, 重複剛剛的動作。若遞迴至第24個frame都成功, 即代表找到可行解; 若frame X在遍歷完所有排列組合後都找不到可行解, 則退回到frame X-1, 並根據剛剛的快照回溯發電機組數據。
 - 發電控制: 發電機控制採用貪婪策略, 使用順序為低成本的綠電>低成本發電機>高成本發電機>電池。發電決策前會先經過state machine的可行性判定, 以確保不會觸犯processor settings。
 - 套利 (**Power_tracer**): 透過注水演算法, 建立當前時段的供電源水桶 (優先序: 綠電 > 運轉中的火電 > 電池放電), 並依序注水至活躍任務的電量需求杯中。所有任務杯子裝滿後, 水桶內剩餘的算力將全數流向市場轉換為售電量。
 - 保留策略: 演算法會計算出每個小時的Slack Capacity (火力機組的最大可能發電量 + 電池剩餘可用放電量), 用於後續的准入控制。
 - 優缺點: DFS與貪婪演算法能用很低的時間複雜度快速找到可行解。但缺點很明顯: 盡管我們已經事先知道Periodic Task的分布, DFS不會嘗試找出全局最佳解。若要找到全局最佳解可能需要使用到混合整數規劃 (MIP) 的技術。
 - 實際效果:
 - 十組測資Hard & Soft deadline miss rate皆為0%

- 十組測資Sporadic Value Rate皆達到100%, 且 Post-acceptance Violation Rate始終保持為 0%
- 准入控制 (**acceptance_tester/main_scheduler**): 當Aperiodic & Sporadic task 抵達時, 准入控制會查詢當前機組的slack capacity判斷是否接受 該任務
 - 實作細節
 - **Sporadic**: 系統會在其生命週期區間內掃描 Slack Array:
 - **Preemptive** 任務: 只要區間內累積有足夠數量的小時滿足需求即判定准入, 並立即扣除對應的資源佔用。
 - **Non-preemptive** 任務: 確保必須找到連續e_j個小時均滿足需求才予放行。若無法滿足上述條件, 直接拒絕。
 - **Aperiodic**: 實作預審查與動態佇列機制
 - 預審查: 任務抵達時, 先檢視從釋放時間到系統邊界之間, 是否有足夠的理論空間。若連理論上都無法完工, 直接拒絕。
 - 佇列推進: 進入佇列的任務, 會在每一個小時嘗試取得當前剩餘 Slack 進行執行。若當前小時算力不足, 則留在佇列中等待。
 - 超時丟棄機制: 為巨型任務導致佇列阻塞, 我們設計了兩道防線:
 - 等候超時: 在佇列中等待超過 24 小時的任務, 強制移出。
 - Impossibility: 若任務的剩餘執行時間已經大於剩餘小時數, 系統會立即將其標記為 Soft Deadline Miss 並丟棄。
- 目標函數權衡:
 - 目標 1: 最小化 **aperiodic miss deadline** 數量
 - 目標 2: 最小化傳統機組發電成本
 - 目標 3: 最大化售電收益
 - 權衡:

■ 套利收益 (f_3) vs. 違規風險 (f_1) :

追求極致的售電利潤會導致發電機滿載，徹底抹殺系統應對突發負載的餘裕，進而引發 Aperiodic Miss 懲罰。我們採取「防禦優先於套利」的貪婪策略，僅在滿足內部負載保底基載後，才將剩餘火力餘裕用於高價套利，確保 f_1 懲罰發生的機率降至最低。(盡管刻意放棄Aperiodic Miss有可能帶來更多獲益)

■ 發電成本 (f_2) vs. 違規風險 (f_1):

極度壓縮發電機出力雖能最小化 f_2 成本，但會導致系統喪失向上爬升的能力，無法接納突發巨型任務。DFS 產出的基底排程確保了機組即使處於低載，也維持著不斷鍊的運轉狀態，使得線上准入控制能隨時運用機組固有的 Ramp-up 空間。

排程演算法設計說明：進階動態排程

Phase 1: 預見與戰略決策

此階段負責環境感知與危機預防，決定系統的宏觀防禦策略：

- 再生能源偏差估測：比對該時段之綠電預測值與受隨機誤差干擾後之實際值，計算系統供需缺口 (ΔP)。
- 缺電應變預案 (當 $\Delta P > 0$ 時觸發)：儲能救援邊界估測：在嚴格遵守「逆變器上限」與「剩餘化學能轉換極限」的雙重物理約束下，計算電池群的最大可用放電量。
- 火力救援邊界估測：依據離線排程演算法給出之合法邊界，評估所有已開機火力機組之剩餘爬升空間。
- 彈性負載卸載：若極限救援加總仍不足彌補缺口，系統將啟動防禦機制，依序中斷執行中的 Aperiodic 任務，並將其狀態凍結退回等候佇列，以此絕對確保 Periodic/Sporadic 任務的存活。
- 溢載吸收預案 (當 $\Delta P < 0$ 時觸發)：估算電池群之剩餘充電容量，作為後續全網能量吸收之依據。

Phase 2: 全域資源配置與市場結算

此階段負責物理層面的實體調度，融合 Phase 1 之戰略決策與即時電力市場訊號：

- 淨負載基準確立：計算扣除實際綠電後之真實淨負載 (W_{net})。為確保系統穩定性與遵守起停約束，所有活躍之火力機組優先指派至其合法邊界之下限。
- 火力調度：依據發電成本遞增排序，平滑調升火力機組出力以覆蓋 W_{net} 。此流程吸收了 Phase 1 預估之火力救援需求，且數學上保證不違反實體機組之 Minimum Up/Down Time 限制。
- 主動高價套利：本實作之關鍵營利機制。若當下即時市價高於某機組之邊際發電成本，排程器將推翻保守配置，強制將該機組推升至合法邊界之上限，主動創造超額電量以傾銷至電網套利。
- 儲能放電決策：若歷經火力調度與套利後， W_{net} 仍存在未被滿足之實體缺口，系統將依據 Phase 1 確立之雙重極限正式啟動電池放電，進行最終補水。

- 全域剩餘電能儲存：結算「綠電 + 火力發電 + 放電 - 系統總負載」後，若全網存在剩餘供電，系統將無差別地將其全數灌入電池充電。此設計確保不論是綠電超發或高價套利之剩餘電能，皆能達成零浪費。
- 能量流溯源與市場結算：執行拓撲能量流追蹤，確認最終注入電網之實際售電量 (Sell t)。最後，計算未達日前承諾之缺口違約金，以及超額售電之即時套利收入，完成 Level 2 目標函數更新。

效能分析：日前排程

為了評估 Level 1 日前排程在不同工作負載情境下的穩定性，我們使用 10 組 scenario 的 `evaluation_results_summary.json` 作為分析資料來源。Level 1 的主要目標是先確保 hard deadline jobs 的正確性，也就是 periodic jobs 與 accepted sporadic jobs 都必須在 deadline 前完成；接著再觀察 aperiodic jobs 的 soft deadline 表現、sporadic jobs 的完成比例，以及 generator cost、market revenue 和 objective value 等經濟效益指標。下表整理 10 組 scenario 的平均值、最小值與最大值，用來呈現 Level 1 日前排程在不同任務分布下的整體效能。

類別	指標	平均值	最小值	最大值	分析重點
Deadline correctness	hard_deadline_miss_rate	0.0000	0.0000	0.0000	所有 periodic jobs 與 accepted sporadic jobs 都準時完成
Deadline correctness	soft_deadline_miss_rate	0.0000	0.0000	0.0000	所有 aperiodic jobs 都在 soft deadline 前完成
Online jobs	sporadic_value_rate	1.0000	1.0000	1.0000	所有 sporadic jobs 都被成功完成

Cost	generator_cost	269,792.12	231,796.00	351,928.40	傳統機組發電成本
Revenue	market_revenue	279,022.62	241,057.20	338,976.20	售電收益
Objective	objective_value	-9,230.50	-21,103.80	12,952.20	10 個 scenario 中有 9 個 objective 為負, 代表售電收益大多高於發電成本

從 Level 1 的 10 組 scenario 結果可以看出, hard_deadline_miss_rate 與 soft_deadline_miss_rate 皆維持為 0, 代表日前排程能穩定滿足 periodic jobs、accepted sporadic jobs 的 hard deadline, 同時 aperiodic jobs 也都能在 soft deadline 前完成。Sporadic value rate 在所有 scenario 中皆為 1.0, 表示 sporadic jobs 全部成功完成, 且 acceptance test 沒有造成 hard deadline violation。

在成本與收益方面, 10 組 scenario 的平均 generator cost 為 269,792.12, 平均 market revenue 為 279,022.62, 因此平均 objective value 為 -9,230.50。由於 Level 1 的 objective 為 $\text{generator_cost} - \text{market_revenue} + \text{soft miss penalty}$, 而 soft deadline miss 為 0, 所以 objective 為負表示售電收益高於發電成本。10 組 scenario 中只有 scenario 10 的 objective 為正, 其餘 9 組皆為負, 代表在多數情境下, 系統能在滿足 deadline constraints 的同時, 透過售電降低整體成本。

Scenario 類	數量	平均	平均	平均	分析
------------	----	----	----	----	----

型		generator_c ost	market_rev enue	objective_v alue	
uniform	3	249,864.27	261,382.20	-11,517.93	均勻分布下 收益穩定高 於成本
concentrate d	3	261,330.67	267,023.40	-5,692.73	集中型任務 造成部分時 段壓力較高 ，收益優勢 較小
mixed	4	291,084.10	301,252.35	-10,168.25	mixed 情境 成本與收益 都較高，但 平均仍有淨 收益

若依照 scenario 類型分析，uniform、concentrated 和 mixed 三類情境的平均 objective value 皆為負，表示三種工作負載分布下，scheduler 都能在滿足 deadline constraints 的前提下取得淨收益。其中 concentrated 情境的平均 objective value 較接近 0，代表當 sporadic / aperiodic jobs 較集中時，系統可利用的 slack 較少，經濟效益相對下降；mixed 情境雖然 generator cost 較高，但 market revenue 也較高，因此平均仍維持負 objective。

效能分析：進階動態排程方法

為了評估 Level 2 進階動態排程在不確定環境下的表現，我們根據 `evaluation_results.json` 中的結果整理主要效能指標。除了 Level 1 原本的 deadline miss rate、response time、tardiness、generator cost 與 market revenue 外，Level 2 額外加入 renewable uncertainty、battery degradation cost、market commitment penalty 與 realtime extra revenue 等指標。這些指標可以用來觀察系統在實際再生能源出力偏離預測值時，是否仍能維持 hard deadline correctness，並評估進階動態排程對成本與收益的影響。整理結果如下表所示。

類別	指標	數值	分析重點
Deadline correctness	hard_deadline_miss_rate	0.0000	所有 hard jobs 準時完成
Deadline correctness	soft_deadline_miss_rate	0.0000	所有 aperiodic jobs 也準時完成
Deadline correctness	average_tardiness / max_tardiness	0.0000 / 0	沒有任何逾期
Online jobs	sporadic_value_rate	1.0000	6 個 sporadic jobs 全部接受且準時完成
Online jobs	post_acceptance_violation_rate	0.0000	acceptance test 沒有造成後續違規
Response	average_response_time / max_response_time	1.1404 / 5	大多數 jobs 在釋放後短時間內完成

Stability	completion_time_jitter	20.5369	periodic completion time 仍有一定變動
Renewable uncertainty	shortfall / surplus events	19 / 20	實際 renewable 有明顯波動
Renewable uncertainty	shortfall / surplus energy	41.7370 / 41.3940	shortfall 與 surplus 總量接近
Battery	battery_degradation_cost	0.0000	本次battery rescue 為0, 因此無老化成本
Market	market_penalty	77.6680	有日前承諾不足造成的小額罰款
Market	realtime_extra_revenue	22,428.3350	即時市場額外收益明顯大於罰款
Cost / Revenue	generator_cost	232,362.5711	傳統機組成本
Cost / Revenue	market_revenue	237,968.8461	售電收益高於發電成本
Objective	objective_value	-5,606.2749	原始 objective 為負, 代表收益抵消成本
Objective	level2_adjusted_objective_value	-27,956.9419	加入 Level 2 市場機制後效益更好

從 Level 2 的評估結果可以看出, hard_deadline_miss_rate 與 soft_deadline_miss_rate 皆為 0, 代表在加入 renewable uncertainty、battery model 與 market commitment 後, scheduler 仍能保護所有 hard deadline jobs, 且 aperiodic jobs 也沒有逾期。Sporadic value rate 為 1.0, 表示 6 個 sporadic jobs 全部被接受並在

deadline 前完成, 且 post_acceptance_violation_rate 為 0, 代表 acceptance test 沒有破壞既有 hard jobs 的可行性。

在 Level 2 動態部分, 系統共發生 19 次 renewable shortfall events 與 20 次 renewable surplus events, 顯示實際再生能源出力確實與 forecast 有偏差。不過本次模擬中 deferred_aperiodic_jobs 為 0, 表示 shortfall 並未嚴重到需要延後 soft jobs。市場方面, 雖然 market commitment shortfall 造成 77.668 的 penalty, 但 realtime_extra_revenue 達到 22,428.335, 明顯高於罰款, 因此 Level 2 adjusted objective value 從原本的 -5,606.2749 下降到 -27,956.9419, 表示加入即時市場收益後整體效益更好。

類別	指標	Level1	Level2	分析重點
Deadline correctness	hard_deadline_miss_rate	0.0000	0.0000	兩者都能保護 hard deadline jobs
Deadline correctness	soft_deadline_miss_rate	0.0000	0.0000	兩者皆無 aperiodic deadline miss
Online jobs	sporadic_value_rate	1.0000	1.0000	sporadic jobs 皆能準時完成
Cost	generator_cost	269,792.12	232,362.57	Level 2 此測資下發電成本較低
Revenue	market_revenue	279,022.62	237,968.85	Level 1 平均售電收益較高
Objective	objective_value	-9,230.50	-5,606.27	兩者原始 objective 皆為負, 代表收益高於成本

Level 2 uncertainty	renewable shortfall / surplus events	N/A	19 / 20	Level 2 額外考 慮 renewable forecast error
Market	market_penalt y	N/A	77.6680	Level 2 有日前 承諾不足罰款
Market	realtime_extra _revenue	N/A	22,428.3350	Level 2 即時市 場帶來額外收 益
Adjusted objective	level2_adjuste d_objective_v alue	N/A	-27,956.9419	加入 Level 2 市場機制後整 體效益更好

由表中可以看出, Level 1 與 Level 2 都能維持 $\text{hard_deadline_miss_rate} = 0$, 表示不論在 deterministic setting 或加入 renewable uncertainty 的情境下, scheduler 都能保護 hard deadline jobs。兩者的 $\text{soft_deadline_miss_rate}$ 也皆為 0, 代表 aperiodic jobs 沒有逾期。

在經濟效益方面, Level 1 的平均 objective value 為 -9,230.50, Level 2 的原始 objective value 為 -5,606.27, 兩者皆為負, 表示 market revenue 高於 generator cost。Level 2 額外考慮 renewable forecast error 與市場承諾機制, 雖然產生 77.668 的 market penalty, 但 realtime extra revenue 達到 22,428.335, 因此 adjusted objective value 下降至 -27,956.9419, 顯示進階動態排程在維持即時正確性的同時, 也能透過即時市場機制提升整體效益。

討論與心得

- **AI輔助策略：**
 - 模型: Gemini 3.1 Pro (Gemini網頁版/Gemini code assistant)、ChatGPT-5.5(/codex agent)
 - workflow: 由於code assistant在處理複雜函式上比較彆扭，我們採用雙模型交互進行的模式 (以下分為AI與Assistant)
 - 擬定框架: 先與AI一同擬定專案框架，將專案細分為數個不同的檔案，以降低Assistant單次生成程式碼的壓力與耦合度。依序從測資生程>資料型別>限制式檢測機>日前排程>准入測試>....，從最底層開始逐一實作各個function。
 - 實作策略: 根據上述解耦後的架構，我們與AI逐一討論各個檔案的實作細節以及技術決策，並由AI負責生程execution prompt。為了確保Assistant有真正理解Spec，我們並沒有將權限全數下放，而是要求Assistant在每次執行前都要先提出執行計畫與To-do list，讓AI與我們一同review，經審核後再開始生成程式碼。
 - 單元測試: 為了在專案尚未成形時就確保各個function的正確性，我們讓AI生程單元測試 (為避免assistant的幻覺，我們讓「單元測試」與「實作」分別由AI與assistant分開實作)，在各個function完成時都執行一次獨立的單元測試，以避免後續橋接各個function時難以判讀問題根源。
 - 結果判讀: 在生成Evaluation results/Schedule result/schedule log後，我們會把結果交由AI進行判讀，判斷中間有沒有矛盾或是可以優化的地方。
 - 人工**Code review**: AI幻覺的第二道防線是人工的Code review，我們在專案進過程中會將程式碼交由現在沒有在開發的人進行code review，以免AI出現幻覺。不過由於程式碼的複雜度較高，code review最多只能做到但一function的綜觀理解，無法深入各個statement的實作細節。

- 總體心得：

- **AI幻覺疏忽**：Demo前我們都只有使用自己本地生成的測資以及Evaluator進行測試，所以直到Demo的時候我們才發現輸出格式有差錯。我們在開發過程中一直都沒有注意到Assistant對於acceptance_test_log、task loading、Evaluation result等格式的認知都是我們在很初期提供的（甚至acceptance_test_log的格式是AI自己幻想出來的），所以後續因為Context window等因素，我們以及AI和assistant都沒有發覺當前的測試腳本是有錯的。從專案完成到Demo期間我們其實都有切換不同的Agent進行結果的雙重確認，但最後還是沒有抓出這個問題，這顯示人工審查在當前的Spec Driven Development (SDD) 中還是有其必要性。
- **本次專案需要改進的地方**：這次專案的團隊合作模式是值得改進的地方，我們的工作流相當陽春：「把功能分成三段，每段一兩個人做，第一段沒做完第二段不能開始」。從開始到結束Github上面都只有一個Branch (main)。這種開發模式在小型、低耦合的專案中還算堪用，但只要專案涉及到能夠（或需要）平行開發的時候，這種開發模式會導致各個開發者的工作流無法pipeline或parallel；大幅拖慢開發進度。
- **AI對專案開發工作流的影響**：大二這屆的同學們算是親身體驗了AI-assisted coding從實用的偷懶小工具演進成比人類還強大的專案實作核心。「正確的」工作流不斷快速地迭代，不免感覺自己的開發模式沒有百分百地運用到AI的開發潛能，但又怕放AI亂跑會在未知的時刻埋下未爆彈。尋求人機協作的平衡，或許是比盲目追求效率更務實的課題。